PromptKit

Composable, version-controlled prompts
for the AI-assisted engineering lifecycle

aka.ms/PromptKit · MIT License · v0.4.0

Engineering Deep-Dive · Getting Started

The Problem

Ad-hoc prompts
Written once, pasted everywhere, never improved

No version control
Can't track what changed, who changed it, or why

No reuse
Every engineer reinvents the same persona / guardrails / format

No testing
No way to verify prompt quality or catch regressions

Inconsistent outputs
Same task → wildly different results across the team

The Fix: Treat Prompts as Code

Modular

Compose from reusable building blocks

Version-Controlled

Track changes via Git like any code

Testable

Reference-compare against known-good

Extensible

Add your own personas, protocols, templates

PromptKit applies the same engineering discipline you use for software to the prompts that build it.

The Engineering Lifecycle

Three composable workflows that cover the full lifecycle of any engineering project

Bootstrap

`spec-extraction-workflow`

Extract a clean semantic baseline from any existing repository: requirements, design, and validation plan

→

Evolve

`engineering-workflow`

Incremental development with human-in-the-loop review. Propagates changes through specs → impl with audits

→

Maintain

`maintenance-workflow`

Periodic health check: re-audit for drift, classify findings, generate corrective patches, restore alignment

↩ Maintain feeds back into Evolve — continuous improvement loop

Domain-agnostic — works for software, hardware, mechanical, RF, protocol engineering
Each workflow is a PromptKit template with full composition: persona + protocols + format

Architecture: The 5-Layer Composition Stack

① Persona — Who the LLM becomes

systems-engineer, security-auditor, devops-engineer ...

② Protocols — How it reasons + guardrails

anti-hallucination, root-cause-analysis, thread-safety ...

③ Format — Output structure & rules

requirements-doc, investigation-report, pipeline-spec ...

④ Taxonomy — Domain classification (optional)

severity levels, priority tiers, risk categories

⑤ Template — The task with parameters

investigate-bug, review-code, author-design-doc ...

Templates declare which components to compose via YAML frontmatter → bootstrap engine snaps them together

What's in the Box

15 Personas

systems-engineer, security-auditor

software-architect, devops-engineer

reverse-engineer, specification-analyst

workflow-arbiter, implementation-engineer

test-engineer, promptkit-contributor

embedded-firmware-engineer, electrical-engineer

rf-engineer, mechanical-engineer, protocol-architect

48 Protocols

Guardrails (5): anti-hallucination, self-verification, operational-constraints

Analysis (17): memory-safety-c, memory-safety-rust, thread-safety, security-vulnerability

Reasoning (26): root-cause-analysis, traceability-audit, code-compliance-audit, requirements-elicitation … and 9 more

21 Formats

requirements-doc, design-doc, validation-plan, investigation-report

implementation-plan, agent-instructions, pipeline-spec, release-notes

multi-artifact, triage-report, promptkit-pull-request

64 Task Templates: investigate-bug · review-code · author-requirements-doc · author-design-doc · audit-traceability · audit-code-compliance · plan-implementation · author-pipeline · triage-issues · extend-library … and more

Quick Start — 3 Commands

Interactive mode

```
npx @alan-jowett/promptkit
```

Detects your LLM CLI (Copilot / Claude) and launches an interactive prompt-building session

Browse templates

```
npx @alan-jowett/promptkit list
```

Lists all available templates with descriptions
Add --json for machine-readable output

Assemble a prompt

```
npx @alan-jowett/promptkit assemble investigate-bug \
  -p problem_description="Segfault on startup" \
  -o investigation.md
```

Composes persona + protocols + format + template
into a single ready-to-use prompt file

Prerequisites: Node.js 18+ · Optional: GitHub Copilot CLI or Claude Code for interactive mode

How Assembly Works

Persona

```
# Identity
You are a senior
systems engineer …
```

→

Protocols

```
# Reasoning
## Anti-
Hallucination
## Root-Cause …
```

→

Format

```
# Output Format
Investigation Report
with sections …
```

→

Template

```
# Task
Investigate the bug:
{{problem_descriptio
n}}
```

Output: Single composed prompt file — ready to paste into any LLM
or use as agent instructions (.github/instructions/*.md, CLAUDE.md, .cursorrules)

Pipeline Chaining: Engineering Workflow

Human-in-the-loop: user reviews and approves after each audit stage.
Adversarial audits use the falsification protocol — actively try to break the output.
Each stage is a full PromptKit template: persona + protocols + format + task.

Get Started Today

Try it now

`npx @alan-jowett/promptkit`

Zero install — just run it. Walks you through template selection and prompt assembly.

Browse the repo

`aka.ms/PromptKit`

Read the templates, protocols, personas. Understand the composition model.

Contribute

`Use the extend-library template`

Add your own personas, protocols, or templates. PromptKit eats its own dog food — the contribution workflow is itself a PromptKit template.

Share feedback

`File issues or start discussions`

What templates are missing? What workflows do you wish PromptKit supported?